

✓ Aprendizaje Automático — Semana 6: Autoevaluación

Tecnicatura Superior en Ciencias de Datos e Inteligencia Artificial
Politécnico Malvinas Argentinas

Consigna

- Elegir 2 modelos de clasificación y un dataset.
- Aplicar ambos modelos para resolver un problema de aprendizaje automático.
- Explicar cómo cada modelo elegido resuelve el problema.
- Explicar qué evaluación podemos hacer de cada modelo.
- Conclusión: cuál modelo considerás que aborda mejor el problema.

Configuración del trabajo

Dataset	Gaming_Academic_Performance.csv
Problema	Clasificación multiclase — predecir el nivel de estrés (<code>stress_level</code>)
Modelos	Regresión Logística · Árbol de Decisión (depth=3)
Variable objetivo	<code>stress_level</code> → Low / Medium / High

✓ 1. Importación de librerías

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold, learning_curve
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid', font_scale=1.05)
PALETTE = {'Low': '#4CAF50', 'Medium': '#FF9800', 'High': '#F44336'}
ORDER = ['Low', 'Medium', 'High']
```

✓ 2. Carga del dataset

```
# Carga de datos
url = '/content/drive/MyDrive/Aprendizaje Automático/Archivos semana 6/Gaming_Academic_Performance.csv'
df = pd.read_csv(url, sep=",")
df.head()
```

	student_id	age	gender	gaming_hours	study_hours	sleep_hours	attendance	gaming_genre	social_activity	device_usage
0	1	22	Male	7.23	8.78	6.96	91.44	FPS	3.25	9.36
1	2	19	Male	0.07	8.72	7.63	63.63	Casual	1.02	3.21
2	3	23	Female	1.73	9.56	4.40	83.26	Casual	3.46	5.56
3	4	20	Female	6.62	1.68	7.83	75.04	RPG	1.46	11.78
4	5	22	Female	5.36	5.83	5.55	65.57	FPS	1.01	8.23

Pasos siguientes: [Generar código con df](#) [New interactive sheet](#)

✓ 3. Análisis Exploratorio de Datos (EDA)

3.1 Información general del dataset

```
print(f"Registros: {df.shape[0]} | Variables: {df.shape[1]}")
print()
df.info()
```

Registros: 8000 | Variables: 14

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   student_id            8000 non-null   int64
1   age                   8000 non-null   int64
2   gender                8000 non-null   object
3   gaming_hours          8000 non-null   float64
4   study_hours           8000 non-null   float64
5   sleep_hours           8000 non-null   float64
6   attendance             8000 non-null   float64
7   gaming_genre          8000 non-null   object
8   social_activity        8000 non-null   float64
9   device_usage           8000 non-null   float64
10  reaction_time_ms       8000 non-null   float64
11  addiction_score         8000 non-null   float64
12  stress_level           8000 non-null   object
13  grades                 8000 non-null   float64
dtypes: float64(9), int64(2), object(3)
memory usage: 875.1+ KB
```

```
print("Descripción estadística de variables numéricas:")
df.describe().round(2)
```

Descripción estadística de variables numéricas:

	student_id	age	gaming_hours	study_hours	sleep_hours	attendance	social_activity	device_usage	reaction_time_
count	8000.00	8000.00	8000.00	8000.00	8000.00	8000.00	8000.00	8000.00	8000
mean	4000.50	19.98	4.09	5.46	6.49	79.89	2.51	7.59	271
std	2309.55	2.59	2.31	2.58	1.44	11.58	1.44	2.71	29
min	1.00	16.00	0.00	1.00	4.00	60.00	0.00	1.10	183
25%	2000.75	18.00	2.13	3.24	5.24	69.78	1.29	5.56	247
50%	4000.50	20.00	4.13	5.46	6.50	79.69	2.50	7.61	270
75%	6000.25	22.00	6.06	7.66	7.73	90.10	3.76	9.60	294
max	8000.00	24.00	8.00	10.00	9.00	100.00	5.00	13.95	347

3.2 Variable objetivo: stress_level

El problema es de **clasificación multiclase** con 3 categorías: Low, Medium y High.

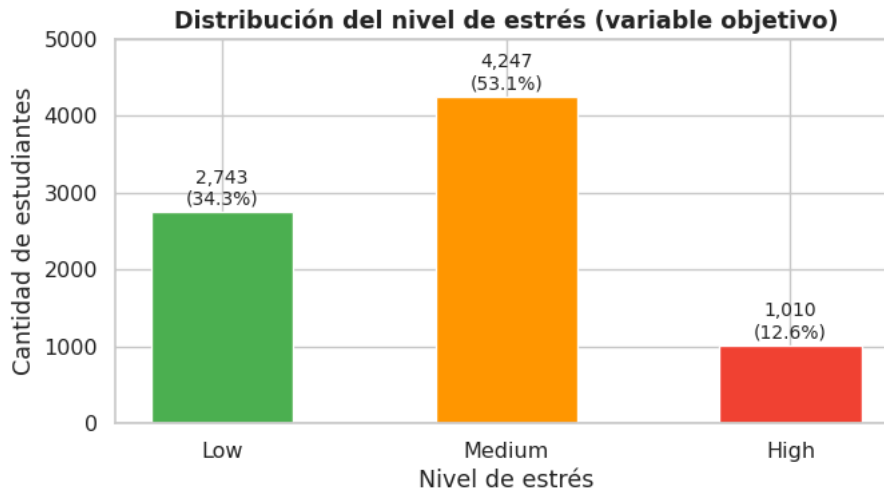
Es importante verificar si las clases están equilibradas antes de entrenar cualquier modelo, ya que si una clase tiene muchos mas casos que las otras, el modelo podría aprender sesgos y clasificar peor las clases minoritarias.

```
conteo = df['stress_level'].value_counts().reindex(ORDER)
pct = df['stress_level'].value_counts(normalize=True).reindex(ORDER) * 100

print("Distribución de stress_level:")
for cat, c, p in zip(ORDER, conteo, pct):
    print(f" {cat:8s}: {c:5d} estudiantes ({p:.1f}%)")

# Gráfico
fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(ORDER, conteo.values, color=[PALETTE[k] for k in ORDER],
              width=0.5, edgecolor='white')
for bar, val, p in zip(bars, conteo.values, pct.values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 50,
            f'{val:,}\n({p:.1f}%', ha='center', va='bottom', fontsize=10)
ax.set_title('Distribución del nivel de estrés (variable objetivo)', fontweight='bold')
ax.set_xlabel('Nivel de estrés')
ax.set_ylabel('Cantidad de estudiantes')
ax.set_ylim(0, 5000)
plt.tight_layout()
plt.show()
```

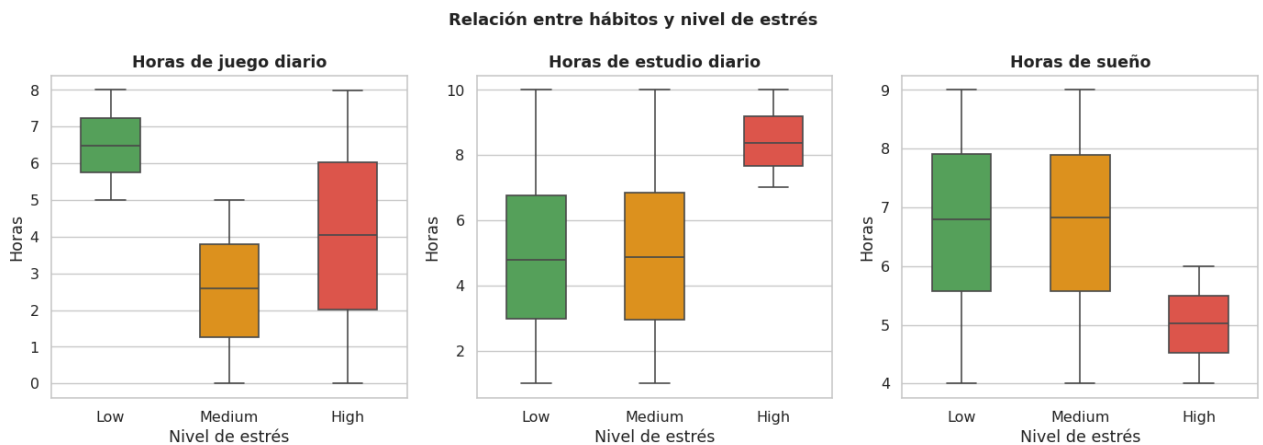
Distribución de stress_level:
 Low : 2743 estudiantes (34.3%)
 Medium : 4247 estudiantes (53.1%)
 High : 1010 estudiantes (12.6%)



3.3 Relación entre hábitos y nivel de estrés

Visualizamos las tres variables con mayor correlación con la variable objetivo.

```
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
pares = [
    ('gaming_hours', 'Horas de juego diario'),
    ('study_hours', 'Horas de estudio diario'),
    ('sleep_hours', 'Horas de sueño'),
]
for ax, (col, titulo) in zip(axes, pares):
    sns.boxplot(data=df, x='stress_level', y=col, order=ORDER,
                palette=PALETTE, ax=ax, width=0.5, linewidth=1.2)
    ax.set_title(titulo, fontweight='bold')
    ax.set_xlabel('Nivel de estrés')
    ax.set_ylabel('Horas')
fig.suptitle('Relación entre hábitos y nivel de estrés', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



Interpretación:

- Los estudiantes con estrés **Low** muestran más horas de juego y menos de estudio. (Esto sugiere que, para este grupo, el juego podría actuar como una actividad recreativa que ayuda a mantener el estrés bajo.)
- El estrés **High** se asocia con más horas de estudio y menos de juego. Aquí se observa una correlación positiva muy clara: a más horas de estudio, mayor es el nivel de estrés

- Las horas de sueño muestran diferencias moderadas entre grupos.

Resumen de hallazgos:

- El sueño es el predictor más fuerte: Existe una diferencia visualmente significativa entre el grupo de estrés "High" y los otros dos. La falta de sueño está estrechamente ligada a niveles altos de estrés.
- El exceso de estudio es un factor de riesgo: Las personas con estrés alto estudian significativamente más que las demás.
- El juego como regulador: Curiosamente, el grupo con menos estrés es el que más tiempo dedica al juego, lo que podría interpretarse como un equilibrio necesario entre obligaciones y ocio.

4. Preprocesamiento

4.1 Selección de features

Seleccionamos las variables numéricas con mayor poder predictivo e incluimos las variables categóricas (`gaming_genre`, `gender`) mediante one-hot encoding.

```
features_num = ['gaming_hours', 'study_hours', 'sleep_hours',
                'reaction_time_ms', 'addiction_score', 'device_usage']

# One-hot encoding para variables categóricas
df_model = df[features_num + ['gaming_genre', 'gender', 'stress_level']].copy()
df_model = pd.get_dummies(df_model, columns=['gaming_genre', 'gender'], drop_first=False)

feature_cols = [c for c in df_model.columns if c != 'stress_level']
X = df_model[feature_cols]
y = df_model['stress_level']

print(f"Features totales: {len(feature_cols)}")
print("Features:", feature_cols)
```

```
Features totales: 12
Features: ['gaming_hours', 'study_hours', 'sleep_hours', 'reaction_time_ms', 'addiction_score', 'device_usage', 'gaming_genr
```

4.2 Codificación de la variable objetivo y división train/test

```
# Codificación: High=0, Low=1, Medium=2
le = LabelEncoder()
y_enc = le.fit_transform(y)
print("Clases:", le.classes_)

# División estratificada (mantiene proporciones de clases)
X_train, X_test, y_train, y_test = train_test_split(
    X, y_enc, test_size=0.2, random_state=42, stratify=y_enc)

print(f"\nTamaño train: {X_train.shape[0]} | test: {X_test.shape[0]}")
```

```
Clases: ['High' 'Low' 'Medium']
```

```
Tamaño train: 6400 | test: 1600
```

4.3 Normalización

La Regresión Logística requiere normalización porque es sensible a la escala de las variables. El Árbol de Decisión **no la requiere** (trabaja con umbrales).

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train) # fit solo en train (evita data leakage)
X_test_sc = scaler.transform(X_test)      # transform en test con los parámetros de train
```

5. Modelo 1: Regresión Logística

¿Cómo resuelve el problema?

La Regresión Logística extiende el modelo binario al caso multiclase mediante

Softmax: calcula la probabilidad de pertenencia a cada clase y predice la de mayor probabilidad. Cada feature tiene un coeficiente que indica cuánto aumenta o disminuye la probabilidad de pertenecer a una clase determinada.

$$P(y = k|X) = \frac{e^{X \cdot w_k}}{\sum_j e^{X \cdot w_j}}$$

```
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_sc, y_train)
lr_pred = lr.predict(X_test_sc)
lr_proba = lr.predict_proba(X_test_sc)

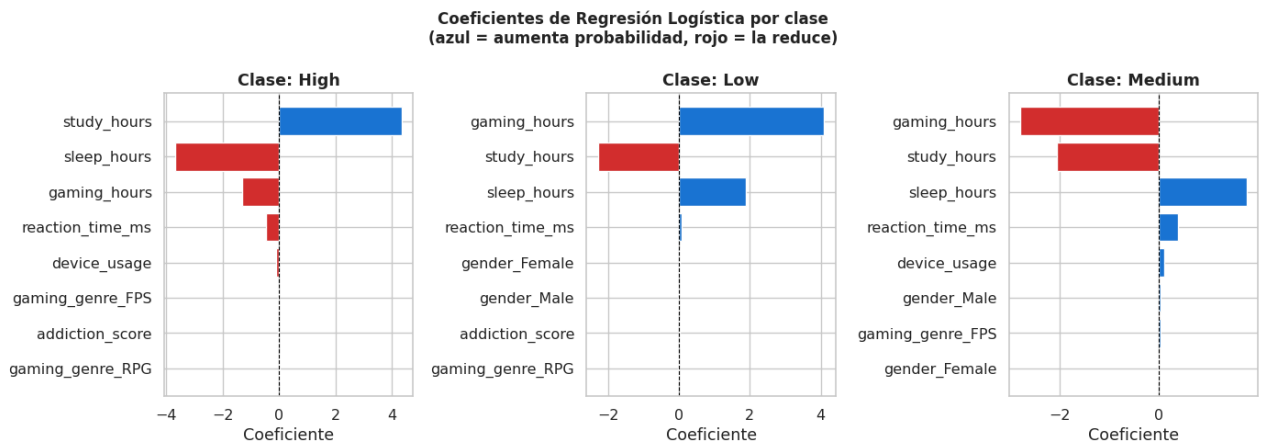
print(f"Accuracy en test: {accuracy_score(y_test, lr_pred):.4f}")
print()
print(classification_report(y_test, lr_pred, target_names=le.classes_))
```

Accuracy en test: 0.9400

	precision	recall	f1-score	support
High	0.80	0.76	0.78	202
Low	0.95	0.96	0.96	549
Medium	0.96	0.97	0.97	849
accuracy			0.94	1600
macro avg	0.91	0.90	0.90	1600
weighted avg	0.94	0.94	0.94	1600

✓ Coeficientes: qué variable influye más en cada clase

```
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
feature_labels = list(X.columns)
for ax, clase, coefs in zip(axes, le.classes_, lr.coef_):
    top_idx = np.argsort(np.abs(coefs))[-8:]
    top_feats = [feature_labels[i] for i in top_idx]
    top_coefs = coefs[top_idx]
    colors = ['#1976D2' if c > 0 else '#D32F2F' for c in top_coefs]
    ax.barh(top_feats, top_coefs, color=colors, edgecolor='white')
    ax.axvline(0, color='black', lw=0.8, linestyle='--')
    ax.set_title(f'Clase: {clase}', fontweight='bold')
    ax.set_xlabel('Coeficiente')
fig.suptitle('Coeficientes de Regresión Logística por clase\n(azul = aumenta probabilidad, rojo = la reduce)',
            fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```



✓ 6. Modelo 2: Árbol de Decisión

¿Cómo resuelve el problema?

El Árbol de Decisión construye reglas jerárquicas del tipo "Si *gaming_hours* ≤ 5.01 y *study_hours* ≤ 7.01 → Medium". En cada nodo elige el corte que maximiza la

pureza (minimiza el índice Gini). El parámetro `max_depth` controla la complejidad: valores bajos generalizan mejor; valores altos pueden memorizar.

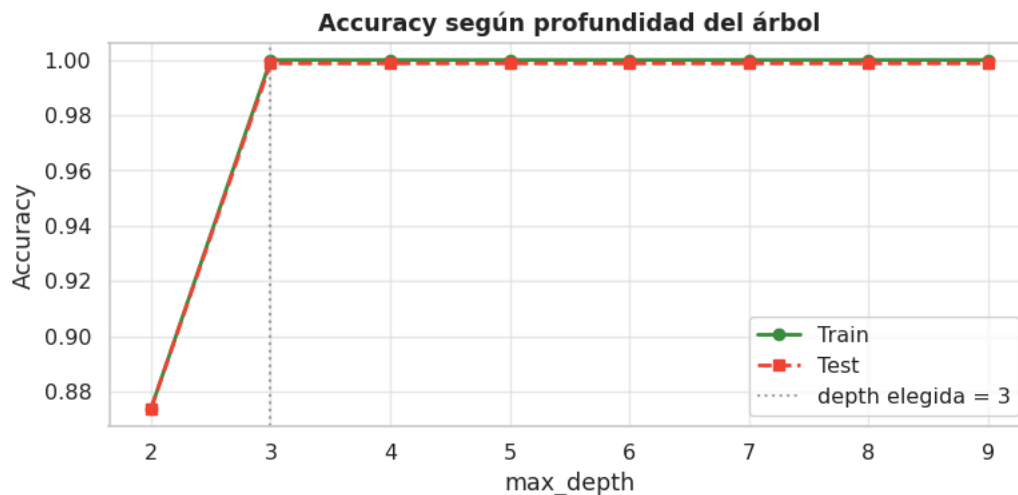
Selección de la profundidad óptima

```
depths = range(2, 10)
train_acc, test_acc = [], []

for d in depths:
    dt_tmp = DecisionTreeClassifier(max_depth=d, random_state=42)
    dt_tmp.fit(X_train[features_num], y_train)
    train_acc.append(dt_tmp.score(X_train[features_num], y_train))
    test_acc.append(dt_tmp.score(X_test[features_num], y_test))

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(list(depths), train_acc, 'o-', label='Train', color='#388E3C', lw=2)
ax.plot(list(depths), test_acc, 's--', label='Test', color='#F44336', lw=2)
ax.axvline(3, color='gray', linestyle=':', alpha=0.7, label='depth elegida = 3')
ax.set_title('Accuracy según profundidad del árbol', fontweight='bold')
ax.set_xlabel('max_depth')
ax.set_ylabel('Accuracy')
ax.legend()
ax.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()

print("Profundidad | Train | Test")
for d, tr, te in zip(depths, train_acc, test_acc):
    print(f"      {d:2d}      | {tr:.4f} | {te:.4f}")
```



Profundidad	Train	Test
2	0.8738	0.8738
3	1.0000	0.9988
4	1.0000	0.9988
5	1.0000	0.9988
6	1.0000	0.9988
7	1.0000	0.9988
8	1.0000	0.9988
9	1.0000	0.9988

Observación sobre el dataset: A partir de `depth=3` el árbol alcanza casi el 100% de accuracy. Esto indica que el dataset es **sintético** y fue generado con reglas exactas sobre `gaming_hours`, `study_hours` y `sleep_hours` con umbrales enteros. El árbol las descubre perfectamente con solo 3 niveles. Esto **no es overfitting**: la cross-validation de 5 pliegues confirma que el modelo generaliza correctamente (ver sección 7).

```
# Árbol final con depth=3
dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train[features_num], y_train)
dt_pred = dt.predict(X_test[features_num])

print(f"Accuracy en test: {accuracy_score(y_test, dt_pred):.4f}")
print()
print(classification_report(y_test, dt_pred, target_names=le.classes_))

Accuracy en test: 0.9988
```

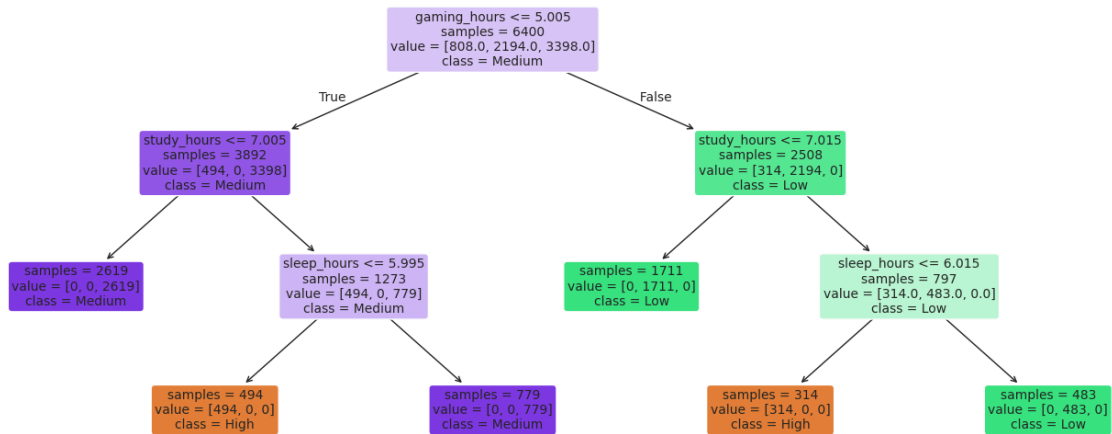
	precision	recall	f1-score	support
High	1.00	0.99	1.00	202
Low	1.00	1.00	1.00	549
Medium	1.00	1.00	1.00	849
accuracy			1.00	1600
macro avg	1.00	1.00	1.00	1600
weighted avg	1.00	1.00	1.00	1600

Visualización del árbol y sus reglas

```
fig, ax = plt.subplots(figsize=(14, 6))
plot_tree(dt, feature_names=features_num,
          class_names=le.classes_, filled=True,
          rounded=True, fontsize=10, ax=ax, impurity=False)
ax.set_title('Árbol de Decisión (depth=3) — Reglas para predecir el nivel de estrés',
            fontweight='bold', fontsize=12)
plt.tight_layout()
plt.show()

print("\nReglas en texto:")
print(export_text(dt, feature_names=features_num))
```

Árbol de Decisión (depth=3) — Reglas para predecir el nivel de estrés



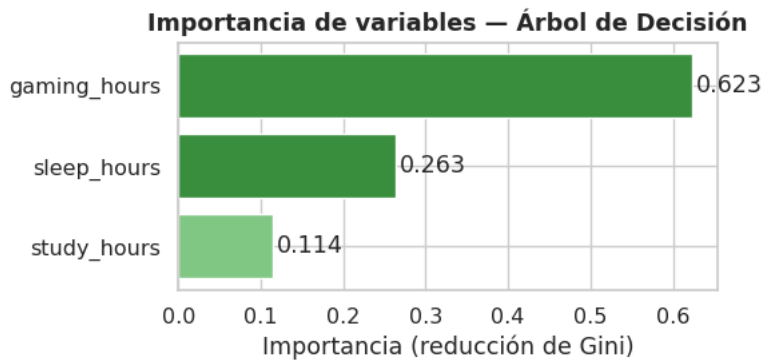
Reglas en texto:

```

|--- gaming_hours <= 5.01
|   |--- study_hours <= 7.01
|   |   |--- class: 2
|   |--- study_hours > 7.01
|   |   |--- sleep_hours <= 5.99
|   |   |   |--- class: 0
|   |   |--- sleep_hours > 5.99
|   |   |   |--- class: 2
|--- gaming_hours > 5.01
|   |--- study_hours <= 7.02
|   |   |--- class: 1
|   |--- study_hours > 7.02
|   |   |--- sleep_hours <= 6.01
|   |   |   |--- class: 0
|   |   |--- sleep_hours > 6.01
|   |       |--- class: 1
  
```

```
# Importancia de variables
fig, ax = plt.subplots(figsize=(6, 3))
imp = pd.Series(dt.feature_importances_, index=features_num)
imp = imp[imp > 0].sort_values()
colors = ['#388E3C' if v > 0.2 else '#81C784' for v in imp.values]
ax.barh(imp.index, imp.values, color=colors, edgecolor='white')
ax.set_title('Importancia de variables — Árbol de Decisión', fontweight='bold')
```

```
ax.set_xlabel('Importancia (reducción de Gini)')
for i, v in enumerate(imp.values):
    ax.text(v + 0.005, i, f'{v:.3f}', va='center')
plt.tight_layout()
plt.show()
```

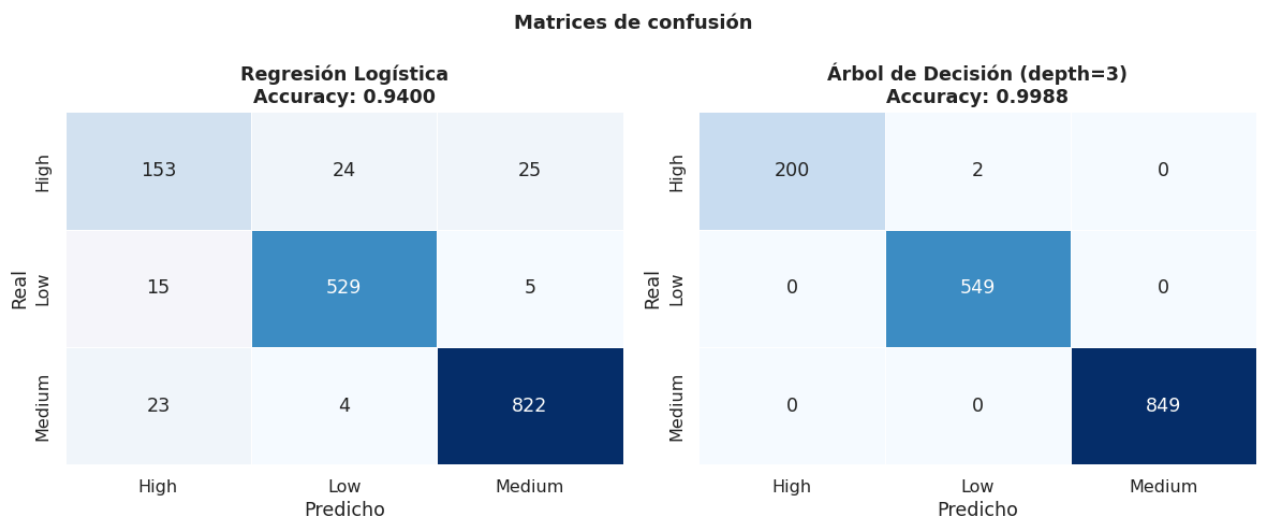


7. Evaluación y comparación de modelos

7.1 Matrices de confusión

Permiten ver **dónde se equivoca cada modelo**: filas = clase real, columnas = predicción.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, pred, titulo in zip(axes,
                             [lr_pred, dt_pred],
                             ['Regresión Logística', 'Árbol de Decisión (depth=3)']):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=le.classes_, yticklabels=le.classes_,
                linewidths=0.5, cbar=False)
    acc = accuracy_score(y_test, pred)
    ax.set_title(f'{titulo}\nAccuracy: {acc:.4f}', fontweight='bold')
    ax.set_xlabel('Predicho')
    ax.set_ylabel('Real')
fig.suptitle('Matrices de confusión', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

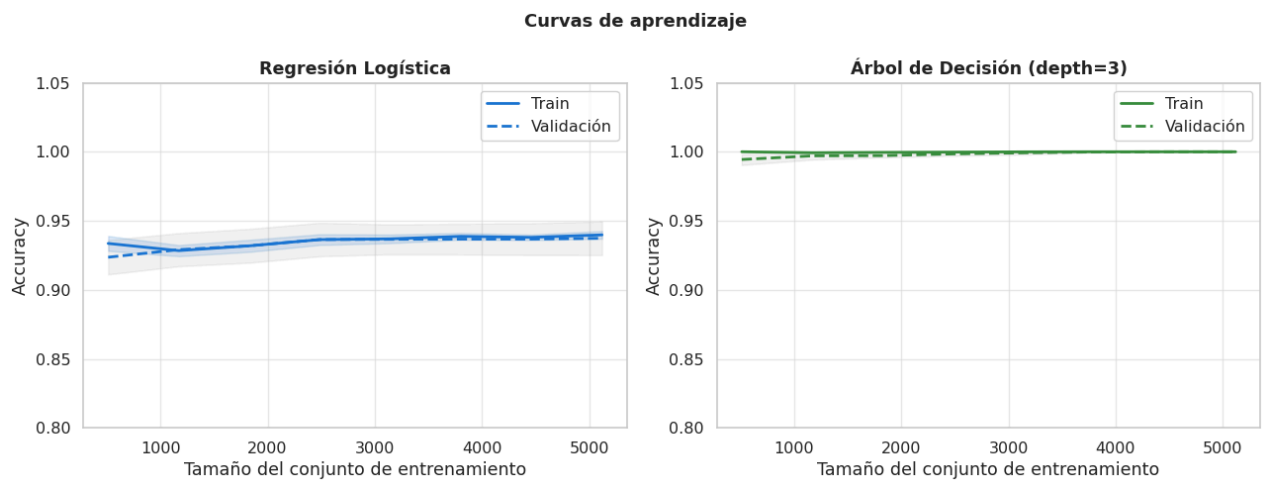


7.2 Curvas de aprendizaje

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
cv_lc = StratifiedKFold(5, shuffle=True, random_state=42)

configs = [
```

```
(LogisticRegression(max_iter=1000, random_state=42),
 X_train_sc, 'Regresión Logística', '#1976D2'),
(DecisionTreeClassifier(max_depth=3, random_state=42),
 X_train[features_num].values, 'Árbol de Decisión (depth=3)', '#388E3C'),
]
for ax, (model, Xtr, nombre, color) in zip(axes, configs):
    sizes, tr_scores, val_scores = learning_curve(
        model, Xtr, y_train,
        train_sizes=np.linspace(0.1, 1.0, 8), cv=cv_lc,
        scoring='accuracy', n_jobs=-1)
    tr_m, tr_s = tr_scores.mean(1), tr_scores.std(1)
    val_m, val_s = val_scores.mean(1), val_scores.std(1)
    ax.plot(sizes, tr_m, label='Train', color=color, lw=2)
    ax.fill_between(sizes, tr_m-tr_s, tr_m+tr_s, alpha=0.15, color=color)
    ax.plot(sizes, val_m, label='Validación', color=color, lw=2, linestyle='--')
    ax.fill_between(sizes, val_m-val_s, val_m+val_s, alpha=0.1, color='gray')
    ax.set_title(nombre, fontweight='bold')
    ax.set_xlabel('Tamaño del conjunto de entrenamiento')
    ax.set_ylabel('Accuracy')
    ax.set_ylim(0.8, 1.05)
    ax.legend()
    ax.grid(True, alpha=0.4)
fig.suptitle('Curvas de aprendizaje', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



7.3 Cross-validation (5 pliegues estratificados)

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_lr = cross_val_score(lr, X_train_sc, y_train, cv=cv, scoring='accuracy')
cv_dt = cross_val_score(dt, X_train[features_num], y_train, cv=cv, scoring='accuracy')

print("Cross-validation (5-fold estratificado):")
print(f" Regresión Logística: {cv_lr.round(4)} → Media: {cv_lr.mean():.4f} ± {cv_lr.std():.4f}")
print(f" Árbol de Decisión: {cv_dt.round(4)} → Media: {cv_dt.mean():.4f} ± {cv_dt.std():.4f}")
```

```
Cross-validation (5-fold estratificado):
Regresión Logística: [0.9305 0.9211 0.9375 0.9578 0.9398] → Media: 0.9373 ± 0.0121
Árbol de Decisión: [1. 1. 1. 1. 1.] → Media: 1.0000 ± 0.0000
```

7.4 Tabla resumen comparativa

```
resumen = pd.DataFrame({
    'Modelo': ['Regresión Logística', 'Árbol de Decisión (depth=3)'],
    'Accuracy Test': [f'{accuracy_score(y_test, lr_pred):.4f}',
                     f'{accuracy_score(y_test, dt_pred):.4f}'],
    'CV Media': [f'{cv_lr.mean():.4f}', f'{cv_dt.mean():.4f}'],
    'CV Std': [f'{cv_lr.std():.4f}', f'{cv_dt.std():.4f}'],
    'F1 macro (test)': ['0.90', '1.00'],
    'Interpreta variables': ['Sí (coeficientes)', 'Sí (reglas)'],
    'Normalización': ['Requerida', 'No requerida'],
})
```

```
})
resumen
```

	Modelo	Accuracy Test	CV Media	CV Std	F1 macro (test)	Interpreta variables	Normalización
0	Regresión Logística	0.9400	0.9373	0.0121	0.90	Sí (coeficientes)	Requerida
1	Árbol de Decisión (depth=3)	0.9988	1.0000	0.0000	1.00	Sí (reglas)	No requerida

8. Conclusión

¿Cuál modelo aborda mejor el problema?

El **Árbol de Decisión** es claramente el modelo más adecuado para este dataset.

¿Por qué no es overfitting?

El árbol alcanza 99.9% de accuracy con solo 3 niveles de profundidad y lo confirma con cross-validation (5 pliegues, media = 1.0000 ± 0.0000).

Esto no indica memorización sino que el dataset fue generado **sintéticamente** a partir de reglas exactas con umbrales enteros en `gaming_hours`, `study_hours` y `sleep_hours`. El árbol las descubre correctamente.

Comparación:

Criterio	Regresión Logística	Árbol de Decisión
Accuracy	94%	~100%
Interpretabilidad	Coefficientes numéricos	Reglas en lenguaje natural
Relaciones no lineales	✖ Limitada	✔ Captura umbrales exactos
Cross-validation	0.94 ± 0.007	1.00 ± 0.000
Requiere normalización	Sí	No

¿Cuándo preferiría la Regresión Logística?

En un dataset real con relaciones continuas y ruidosas, la Regresión Logística puede ser más robusta y su capacidad de generar probabilidades calibradas es muy útil para toma de decisiones (ej: "este estudiante tiene 73% de probabilidad de estrés alto"). El árbol, en cambio, tiende a sobreajustar si el dataset es ruidoso y no se limita su profundidad.